

Numerical Analysis

Chapter 2: Root Finding of Nonlinear Equations

Prof. See Jo Kim

Department of Mechanical & Robotics Engineering

(2.1) Nonlinear Root-Finding Problem

We seek the solution of a nonlinear equation

$$f(x) = 0 \quad (2.1-1)$$

Such equations frequently appear in engineering and science, and analytical solutions are often unavailable.

Examples of Nonlinear Equations

$$f(\theta) = \theta^3 + \cos \theta \quad (2.1-2)$$

- Polynomial + trigonometric term
- Multiple roots may exist
- Numerical methods are required

Scalar and Vector Nonlinear Systems

Scalar equation:

$$f(x) = 0$$

Vector system:

$$\mathbf{f}(\mathbf{x}) = \mathbf{0} \quad (2.1-3)$$

(2.2) Incremental Search Method

The incremental search method detects sign changes of $f(x)$.

$$f(x_i)f(x_{i+1}) < 0$$

This condition guarantees a root in (x_i, x_{i+1}) .

Algorithm: Incremental Search

- 1 Choose initial interval $[a, b]$
- 2 Select step size Δx
- 3 Evaluate $f(x)$ sequentially
- 4 Detect sign change
- 5 Narrow the interval

Python Code: Incremental Search

```
def incremental_search(f, a, b, dx):  
    x = a  
    while x < b:  
        if f(x) * f(x + dx) < 0:  
            return x, x + dx  
        x += dx  
    return None
```

Code Explanation: Incremental Search

- f : target nonlinear function
- a, b : search interval
- dx : incremental step size
- $f(x) * f(x+dx) < 0$: sign change detection
- Smaller dx improves accuracy but increases cost

Graphical Interpretation

incremental_search.png

(2.3) Bisection Method

The bisection method is a bracketing method with guaranteed convergence.

$$f(a)f(b) < 0$$

- Start from interval $[a, b]$
- Compute midpoint

$$c = \frac{a + b}{2}$$

- Select subinterval containing the root

Algorithm: Bisection Method

- 1 Choose a, b such that $f(a)f(b) < 0$
- 2 Compute midpoint c
- 3 Check sign of $f(c)$
- 4 Update interval
- 5 Repeat until convergence

Convergence Property

- Guaranteed convergence
- Linear convergence
- Error bound:

$$|e_n| \leq \frac{b - a}{2^n}$$

Python Code: Bisection Method

```
def bisection(f, a, b, tol=1e-6):  
    while abs(b - a) > tol:  
        c = (a + b) / 2.0  
        if f(a) * f(c) < 0:  
            b = c  
        else:  
            a = c  
    return (a + b) / 2.0
```

Code Explanation: Bisection Method

- a, b : bracketing interval
- tol : stopping tolerance
- Interval width controls accuracy
- Robust but slow convergence

Graphical Illustration

bisection_method.png

(2.4) Newton–Raphson Method

Based on Taylor expansion:

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n) \quad (2.4-1)$$

Iteration Formula

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (2.4-2)$$

Algorithm: Newton–Raphson Method

- 1 Choose initial guess x_0
- 2 Evaluate $f(x_n)$ and $f'(x_n)$
- 3 Update x_{n+1}
- 4 Check convergence
- 5 Repeat

Python Code: Newton–Raphson

```
def newton_raphson(f, df, x0, tol=1e-6, max_iter=100):  
    x = x0  
    for i in range(max_iter):  
        if abs(f(x)) < tol:  
            return x  
        x = x - f(x)/df(x)  
    raise RuntimeError("No convergence")
```

Code Explanation: Newton–Raphson

- df: derivative of $f(x)$
- Quadratic convergence near root
- Sensitive to initial guess
- May fail if $f'(x) = 0$

Graphical Interpretation

newton_raphson.png

Comparison of Methods

Method	Speed	Stability	Derivative
Incremental Search	Slow	High	No
Bisection	Slow	Very High	No
Newton–Raphson	Fast	Low	Yes

Key Takeaways

- Root-finding is fundamental in numerical analysis
- Bracketing methods are robust
- Newton method is fast but sensitive